🎓 **Exam Type: Practical**
⏳ **Duration: 6 Hours**

Deep Learning

Project Overview

You are a Junior Deep Learning Engineer at a mid-sized Indian tech company. Over the next 6 hours you will build five standalone mini-projects — one per core Deep Learning topic — using real-world or publicly available datasets. Each mini-project is self-contained: you load data, build a model, train it, evaluate it, and save the result.

This is a basic-level exam. You are not expected to achieve state-of-the-art accuracy. You are expected to write clean, working code, explain what each component does, and demonstrate that you understand why you made each design choice.

📋 **Topics covered in this exam:**

Step 1 — ANN: Artificial Neural Network for tabular classification

Step 2 — OpenCV: Image preprocessing and feature extraction pipeline

Step 3 — CNN: Convolutional Neural Network for image classification

Step 4 — RNN: Recurrent Neural Network for sequence prediction

Step 5 — Transformers: Text classification with Hugging Face

Step 6 — Comparison, reflection, and GitHub submission

🎯 Learning Objectives

By completing this exam, students will demonstrate:

- Building, training and evaluating an ANN on structured tabular data using Keras.
- Using OpenCV to load, resize, convert, denoise and extract basic features from images.
- Designing a simple CNN architecture for image classification and interpreting training curves.
- Building an RNN (SimpleRNN or LSTM) for a sequence prediction task and evaluating it.
- Using a pre-trained Hugging Face Transformer pipeline for text classification without fine-tuning.
- Saving trained models and writing clean, reproducible notebooks pushed to GitHub.

🧠 Step 1: ANN — Artificial Neural Network

📁 **Dataset: Pima Indians Diabetes Dataset**

🔗 Kaggle: https://www.kaggle.com/datasets/kumargh/pimaindiansdiabetescsv

📊 Rows: 768 patients | Features: 8 medical inputs | Target: Outcome (0 = No diabetes, 1 = Diabetes)

🎯 Task: Predict whether a patient has diabetes — binary classification

1.1 Data Loading & Preprocessing

- Load diabetes.csv using pandas. Print .shape, .head(5), and check for missing values.
- Replace zero values in Glucose, BloodPressure, SkinThickness, Insulin, BMI with the column median (zeros are physiologically impossible — they represent missing data).
- Split into features (X) and target (y). Apply StandardScaler to X.
- Train-test split: 80% train / 20% test, random_state=42, stratify=y.

1.2 Build the ANN

- Use Keras Sequential API. Build a network with:
  - Input layer matching the number of features
  - Two hidden Dense layers with 16 and 8 neurons, ReLU activation
  - Output Dense layer with 1 neuron, Sigmoid activation (binary output)
- Compile with optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'].
- Print model.summary().

1.3 Train & Evaluate

- Train for 50 epochs with batch_size=32. Use validation_split=0.2 during training.
- Plot training and validation accuracy vs epochs on one figure.
- Plot training and validation loss vs epochs on one figure.
- Evaluate on the test set. Print: Test Accuracy, Precision, Recall, F1-Score, Confusion Matrix.

1.4 Save & Reflect

- Save the trained model: model.save('ann_diabetes_model.h5').
- Write a Markdown cell: What does each layer do? Why did you choose 16 and 8 neurons? What does the Sigmoid output represent?

💡 Target accuracy: aim for 70–80%. If validation loss is rising while training loss falls, your model is overfitting — try adding a Dropout(0.3) layer between the hidden layers.

👁️ Step 2: OpenCV — Image Preprocessing Pipeline

📁 **Dataset: Any 5 images of your choice (flowers, animals, objects)**

🔗 Option A: Download 5 images manually from Google Images or Unsplash

🔗 Option B: Use 5 images from the Flowers Recognition dataset on Kaggle:

https://www.kaggle.com/datasets/alxmamaev/flowers-recognition

📊 Input: 5 raw images (any format: JPG/PNG) | Output: Preprocessed versions of each

🎯 Task: Demonstrate a complete OpenCV preprocessing pipeline — no model training required

2.1 Load & Display

- Read all 5 images using cv2.imread(). Display each using matplotlib (convert BGR → RGB with cv2.cvtColor).
- Print the shape of each image (height, width, channels).

2.2 Preprocessing Operations

For each image, apply all of the following and display the result in a subplot grid:

- Resize to 128×128 pixels using cv2.resize().
- Convert to Grayscale using cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).
- Apply Gaussian Blur (kernel size 5×5) using cv2.GaussianBlur().
- Apply Canny Edge Detection using cv2.Canny(gray, 100, 200).
- Draw a green bounding rectangle around the largest contour found with cv2.findContours().

2.3 Summary

- Create a 5×5 grid: rows = images, columns = [Original, Resized, Grayscale, Edges, Contour].
- Write a Markdown cell explaining what each preprocessing step does and why it is useful before feeding images into a CNN.

💡 Tip: Save all results using cv2.imwrite() or plt.savefig() so they appear in your GitHub repo as evidence.

🖼️ Step 3: CNN — Convolutional Neural Network

📁 **Dataset: CIFAR-10 (built into Keras)**

🔗 Load with: from tensorflow.keras.datasets import cifar10

📊 60,000 images (32×32, RGB) | 10 classes: airplane, car, bird, cat, deer, dog, frog, horse, ship, truck

🎯 Task: Build and train a CNN to classify images into one of 10 classes

⚠️ Note: Use only 20,000 training samples and 4,000 test samples to reduce training time.

3.1 Data Loading & Preparation

- Load CIFAR-10: (X_train, y_train), (X_test, y_test) = cifar10.load_data().
- Slice to 20,000 training samples and 4,000 test samples using array indexing.
- Normalise pixel values to [0, 1] by dividing by 255.0.
- Apply to_categorical() to convert y_train and y_test to one-hot vectors.
- Display 10 sample images in a row with their class labels as titles.

3.2 Build the CNN

- Use Keras Sequential API. Build:
  - Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3))
  - MaxPooling2D(2,2)
  - Conv2D(64, (3,3), activation='relu')
  - MaxPooling2D(2,2)
  - Flatten()
  - Dense(64, activation='relu')
  - Dense(10, activation='softmax') — 10 classes
- Compile with optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'].
- Print model.summary().

3.3 Train & Evaluate

- Train for 10 epochs, batch_size=64, validation_split=0.1.
- Plot accuracy and loss curves (training vs validation) on separate figures.
- Evaluate on the test set. Print Test Accuracy and Classification Report.
- Display 10 test images: show True Label vs Predicted Label as image titles. Highlight wrong predictions in red.

3.4 Save & Reflect

- Save the trained model: model.save('cnn_cifar10_model.h5').
- Write a Markdown cell: What does a Conv2D layer extract? What does MaxPooling do? Why does Flatten come before Dense?

💡 Target accuracy: 55–70% on this small subset is fine for a basic exam. Focus on correct architecture and clean training curves, not peak accuracy.

🔄 Step 4: RNN — Recurrent Neural Network

📁 **Dataset: Monthly Airline Passengers (time-series)**

🔗 Direct CSV:
https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv

📊 Rows: 144 months (Jan 1949 – Dec 1960) | Column: 'Passengers' (monthly count)

🎯 Task: Predict next month's passenger count from the previous 12 months — univariate time-series forecasting

4.1 Data Loading & Preparation

- Load the CSV using pandas. Print .head(5) and plot the full time-series as a line chart.
- Normalise the 'Passengers' column using MinMaxScaler (scale to [0, 1]).
- Create sequences: for each window of 12 consecutive months (look_back=12), the next month is the target. Use a loop to create X and y arrays.
- Reshape X to (samples, timesteps, features) = (n, 12, 1) as required by Keras RNN layers.
- Train-test split: use the first 80% of sequences for training, the remaining 20% for testing (do NOT shuffle — time order matters).

4.2 Build the RNN

- Use Keras Sequential API. Build:
  - SimpleRNN(50, activation='tanh', input_shape=(12, 1))
  - Dense(1) — single value prediction
- Compile with optimizer='adam', loss='mse'.
- Print model.summary().

4.3 Train & Evaluate

- Train for 50 epochs, batch_size=16. No validation_split (data is too small).
- Plot training loss vs epochs.
- Predict on both train and test sets. Inverse-transform predictions back to original passenger count scale using scaler.inverse_transform().
- Plot: Actual passengers vs Predicted (train in blue, test in orange) on a single line chart.
- Report: RMSE (Root Mean Squared Error) on the test set.

4.4 Save & Reflect

- Save the model: model.save('rnn_airline_model.h5').
- Write a Markdown cell: What is a recurrent connection? Why must we not shuffle time-series data before splitting? What is RMSE and what does it tell you about prediction quality?

💡 Tip: If you want to try an LSTM instead of SimpleRNN, simply replace SimpleRNN(50) with LSTM(50). Note whether LSTM gives better or worse results and mention it in your reflection.

🤗 Step 5: Transformers — Text Classification with Hugging Face

📁 **Dataset: IMDb Movie Reviews (25 custom sentences — you write them)**

🔗 Hugging Face model: distilbert-base-uncased-finetuned-sst-2-english

📊 Input: 25 sentences you create (mix of positive, negative, and neutral movie reviews)

🎯 Task: Use a pre-trained Transformer pipeline to classify sentiment — no training required

⚠️ Note: Install with: pip install transformers. No GPU needed for inference-only.

5.1 Setup & Pipeline

- Install and import: from transformers import pipeline.
- Load the sentiment analysis pipeline: classifier = pipeline('sentiment-analysis', model='distilbert-base-uncased-finetuned-sst-2-english').
- Write 25 sentences yourself — a mix of:
  - 10 clearly positive reviews (e.g., 'The film was absolutely brilliant and moving.')
  - 10 clearly negative reviews (e.g., 'Worst movie I have ever seen. Completely boring.')
  - 5 ambiguous or neutral sentences (e.g., 'The acting was decent but the plot was weak.')

5.2 Run Inference & Analyse

- Pass all 25 sentences through classifier(). Print label and score for each.
- Store results in a DataFrame with columns: Sentence, True_Sentiment (you assign: POS/NEG/NEU), Predicted_Label, Confidence_Score.
- Compute and print: how many of the 20 clearly positive/negative sentences did the model predict correctly?
- Print the 5 ambiguous sentences and discuss: did the model lean positive or negative? Does the confidence score reflect the ambiguity?

5.3 Visualise & Reflect

- Plot a horizontal bar chart: sentence index (1–25) on y-axis, confidence score on x-axis. Colour bars green for POSITIVE, red for NEGATIVE.
- Write a Markdown cell covering: What is a Transformer? What does 'pre-trained' and 'fine-tuned' mean? Why can we use this model without any training of our own? What are the limitations of using a pre-trained English sentiment model on Indian-English text?

💡 India context: Try a few sentences written in Indian English (e.g., 'The movie was timepass only.' or 'Total bakwaas picture.'). Note how the model handles these and mention it in your reflection.

📊 Step 6: Comparison, Reflection & GitHub Submission

6.1 Results Summary Table

Create a summary table in a Markdown cell or as a pandas DataFrame showing all five mini-projects:

Step / Topic	ANN, OpenCV, CNN, RNN, Transformer
Task type	Classification / Preprocessing / Classification / Regression / Inference
Dataset used	Name and size of dataset for each step
Key metric	Accuracy / N/A / Accuracy / RMSE / Accuracy (manually checked)
Result achieved	Your actual number for each step
Time taken	Approximate minutes spent on each step

6.2 Written Reflection

Write a Markdown cell (5–8 bullet points) answering:

- Which step was the most challenging and why?
- Which model gave you the best result relative to the effort required?
- What is one thing you would change about your ANN architecture if you had more time?
- Why does the CNN outperform the ANN on image data?
- What is the fundamental difference between how an RNN processes data vs how a Transformer does it?
- What would be the next step to turn your CNN into a production-ready image classifier?

6.3 GitHub Submission

- Create a GitHub repository named: deep-learning-practical-exam-set-a.
- Push the following files:
  - DeepLearning_Practical_Exam_SetA.ipynb (single fully executed notebook)
  - ann_diabetes_model.h5
  - cnn_cifar10_model.h5
  - rnn_airline_model.h5
  - outputs/ folder with OpenCV images (original + preprocessed)
  - summary_report.md (see below)
  - requirements.txt (tensorflow, keras, opencv-python, transformers, pandas, numpy, matplotlib, seaborn, scikit-learn)
  - README.md (project overview, steps covered, how to run, results table)

6.4 Summary Report (summary_report.md)

Write ~400–500 words covering:

- What were the five tasks and which datasets did you use?
- Which Deep Learning concept was new to you, and what did you learn?
- What results did you achieve on each task?

- What would you build next if you had more time and compute?

Marking Scheme (50 Marks)

Total Marks: 50 — distributed across 3 components as follows:

Component	Marks	Deliverable
A. Technical Completion	30 Marks	Completed .ipynb with all five DL tasks built, trained and functional
B. Recorded Video (Face + Screen)	15 Marks	5–10 min MP4/MOV showing face + screen with verbal explanation of each model and results
C. GitHub Repository + README.md	5 Marks	Public GitHub repo with saved models, README, video link, commits
TOTAL	**50 Marks**	

Component B: Recorded Video — Face + Screen (15 Marks)

Record a video showing your face (webcam picture-in-picture) and your computer's entire screen simultaneously. Walk through each step verbally — explain what the model does, show training curves, and interpret results. Do not just click without speaking.

Video Requirements:

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Entire screen (demonstrating .ipynb, training curves, predictions, and visualisations)
Duration	5 to 10 minutes
Naming convention	Practical_Exam_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (5 Marks)

Create a public GitHub repository specifically for this project. Push all files listed in Section 6.3. The README.md must include: project title, list of all five steps, dataset links, how to install requirements and run the notebook, a results summary table, and your video link.

📄 Deliverables

Notebook	DeepLearning_Practical_Exam_SetA.ipynb (fully executed, all outputs and plots visible)
Saved Models	ann_diabetes_model.h5 + cnn_cifar10_model.h5 + rnn_airline_model.h5
OpenCV Outputs	outputs/ folder with preprocessed image results (5 images × 5 operations)
Summary Report	summary_report.md (~1 page, results + reflection)
GitHub Repo	All of the above + requirements.txt + README.md + video link

✅ **Submission Checklist:**

☐ Notebook runs top-to-bottom without errors

☐ Step 1 (ANN): model.summary() printed, accuracy/loss plots shown, model saved as .h5

☐ Step 2 (OpenCV): 5×5 preprocessing grid displayed with all 5 operations

☐ Step 3 (CNN): training curves plotted, 10 test predictions with true/predicted labels shown

☐ Step 4 (RNN): actual vs predicted line chart plotted, RMSE reported

☐ Step 5 (Transformer): 25-sentence DataFrame with confidence scores, bar chart displayed

☐ Step 6: results summary table and written reflection complete

☐ GitHub repo link shared with examiner

☐ Video URL pasted in README.md and shared with examiner

BRING ON YOUR CODING ATTITUDE

Practical Exam — Deep Learning